

Figure 2: t-SNE visualization of audio (speech) representations for a subset of 10 speakers (out of 251). Every color represents a different speaker.

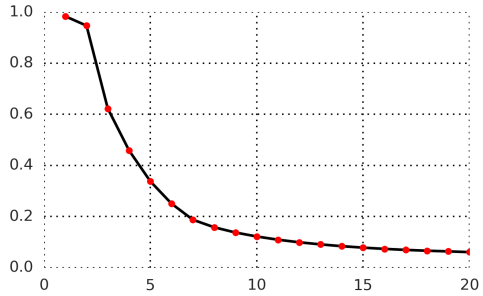



Figure 3: Average accuracy of predicting the positive sample in the contrastive loss for 1 to 20 latent steps in the future of a speech waveform. The model predicts up to 200ms in the future as every step consists of 10ms of audio.

| Method | ACC |
|---|---|
| **Phone classification** | |
| Random initialization | 27.6 |
| MFCC features | 39.7 |
| CPC | 64.6 |
| Supervised | 74.6 |
| **Speaker classification** | |
| Random initialization | 1.87 |
| MFCC features | 17.6 |
| CPC | 97.4 |
| Supervised | 98.5 |

Table 1: LibriSpeech phone and speaker classification results. For phone classification there are 41 possible classes and for speaker classification 251. All models used the same architecture and the same audio input sizes.

| Method | ACC |
|---|---|
| **#steps predicted** | |
| 2 steps | 28.5 |
| 4 steps | 57.6 |
| 8 steps | 63.6 |
| 12 steps | 64.6 |
| 16 steps | 63.8 |
| **Negative samples from** | |
| Mixed speaker | 64.6 |
| Same speaker | 65.5 |
| Mixed speaker (excl.) | 57.3 |
| Same speaker (excl.) | 64.6 |
| Current sequence only | 65.2 |

Table 2: LibriSpeech phone classification ablation experiments. More details can be found in Section 3.1.

with the Kaldi toolkit [31] and pre-trained models on Librispeech[1]. We have made the aligned phone labels and our train/test split available for download on Google Drive[2]. The dataset contains speech from 251 different speakers.

The encoder architecture $g_{enc}$ used in our experiments consists of a strided convolutional neural network that runs directly on the 16KHz PCM audio waveform. We use five convolutional layers with strides [5, 4, 2, 2, 2], filter-sizes [10, 8, 4, 4, 4] and 512 hidden units with ReLU activations. The total downsampling factor of the network is 160 so that there is a feature vector for every 10ms of speech, which is also the rate of the phoneme sequence labels obtained with Kaldi. We then use a GRU RNN [17] for the autoregressive part of the model, $g_{ar}$ with 256 dimensional hidden state. The output of the GRU at every timestep is used as the context $c$ from which we predict 12 timesteps in the future using the contrastive loss. We train on sampled audio windows of length 20480. We use the Adam optimizer [32] with a learning rate of 2e-4, and use 8 GPUs each with a minibatch of 8 examples from which the negative samples in the contrastive loss are drawn. The model is trained until convergence, which happens roughly at 300,000 updates.

Figure 3 shows the accuracy of the model to predict latents in the future, from 1 to 20 timesteps. We report the average number of times the logit for the positive sample is higher than for the negative samples in the probabilistic contrastive loss. This figure also shows that the objective is neither trivial nor impossible, and as expected the prediction task becomes harder as the target is further away.

[1] www.kaldi-asr.org/downloads/build/6/trunk/egs/librispeech/

[2] https://drive.google.com/drive/folders/1BhJ2umKH3whguxMwifaKtSra0TgAbtfb

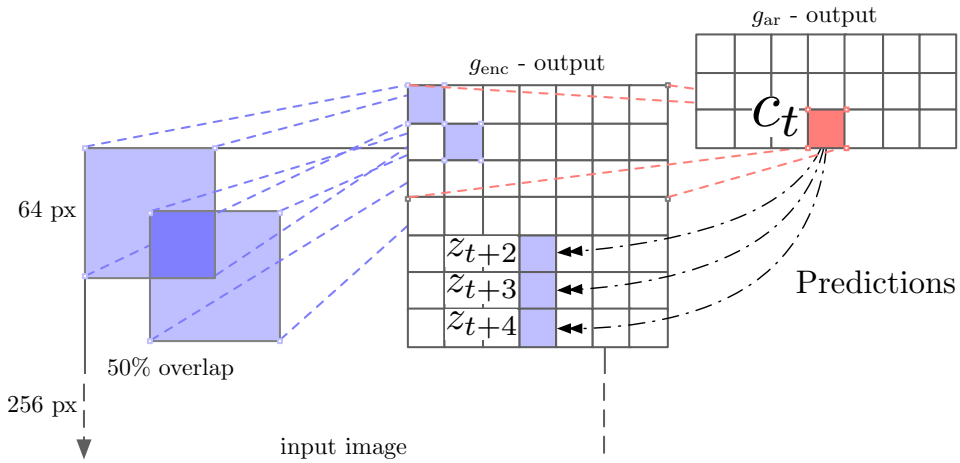



Figure 4: Visualization of Contrastive Predictive Coding for images (2D adaptation of Figure 1).

To understand the representations extracted by CPC, we measure the phone prediction performance with a linear classifier trained on top of these features, which shows how linearly separable the relevant classes are under these features. We extract the outputs of the GRU (256 dimensional), i.e. $c_t$, for the whole dataset after model convergence and train a multi-class linear logistic regression classifier. The results are shown in Table 1 (top). We compare the accuracy with three baselines: representations from a random initialized model (i.e., $g_{enc}$ and $g_{ar}$ are untrained), MFCC features, and a model that is trained end-to-end supervised with the labeled data. These two models have the same architecture as the one used to extract the CPC representations. The fully supervised model serves as an indication for what is achievable with this architecture. We also found that not all the information encoded is linearly accessible. When we used a single hidden layer instead the accuracy increases from **64.6** to **72.5**, which is closer to the accuracy of the fully supervised model.

Table 2 gives an overview of two ablation studies of CPC for phone classification. In the first set we vary the number of steps the model predicts showing that predicting multiple steps is important for learning useful features. In the second set we compare different strategies for drawing negative sample, all predicting 12 steps (which gave the best result in the first ablation). In the mixed speaker experiment the negative samples contain examples of different speakers (first row), in contrast to same speaker experiment (second row). In the third and fourth experiment we exclude the current sequence to draw negative samples from (so only other examples in the minibatch are present in $X$) and in the last experiment we only draw negative samples within the sequence (thus all samples are from the same speaker).

Beyond phone classification, Table 1 (bottom) shows the accuracy of performing speaker identity (out of 251) with a linear classifier from the same representation (we do not average utterances over time). Interestingly, CPCs capture both speaker identity and speech contents, as demonstrated by the good accuracies attained with a simple linear classifier, which also gets close to the oracle, fully supervised networks.

Additionally, Figure 2 shows a t-SNE visualization [33] of how discriminative the embeddings are for speaker voice-characteristics. It is important to note that the window size (maximum context size for the GRU) has a big impact on the performance, and longer segments would give better results. Our model had a maximum of 20480 timesteps to process, which is slightly longer than a second.

### 3.2 Vision

In our visual representation experiments we use the ILSVRC ImageNet competition dataset [34]. The ImageNet dataset has been used to evaluate unsupervised vision models by many authors [28, 11, 35, 10, 29, 36]. We follow the same setup as [36] and use a ResNet v2 101 architecture [37] as the image encoder $g_{enc}$ to extract CPC representations (note that this encoder is not pretrained). We did not use Batch-Norm [38]. After unsupervised training, a linear layer is trained to measure classification accuracy on ImageNet labels.

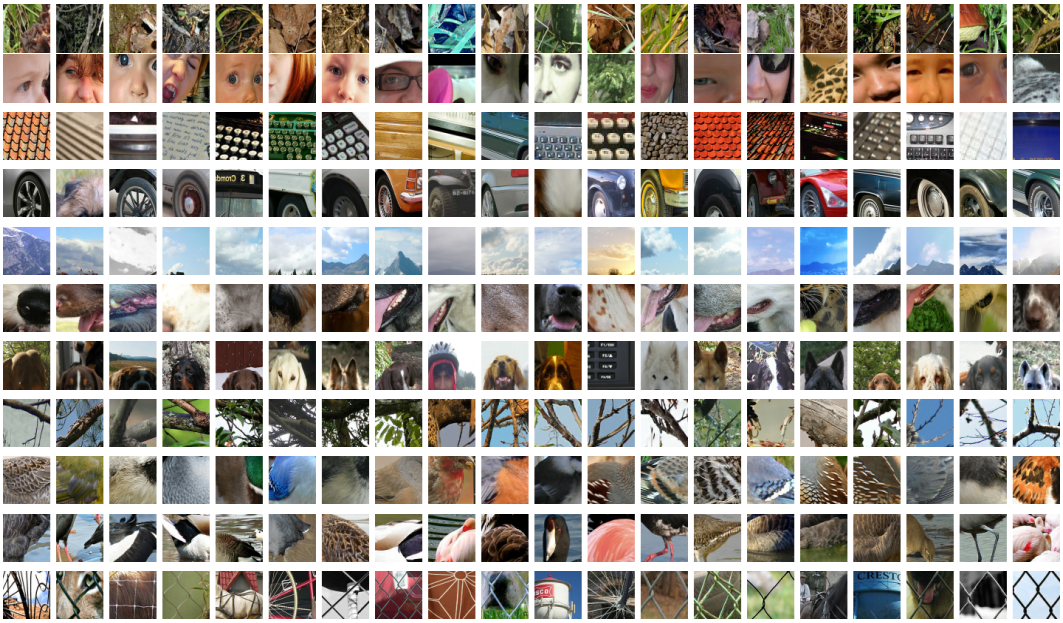


Figure 5: Every row shows image patches that activate a certain neuron in the CPC architecture.

The training procedure is as follows: from a 256x256 image we extract a 7x7 grid of 64x64 crops with 32 pixels overlap. Simple data augmentation proved helpful on both the 256x256 images and the 64x64 crops. The 256x256 images are randomly cropped from a 300x300 image, horizontally flipped with a probability of 50% and converted to greyscale. For each of the 64x64 crops we randomly take a 60x60 subcrop and pad them back to a 64x64 image.

Each crop is then encoded by the ResNet-v2-101 encoder. We use the outputs from the third residual block, and spatially mean-pool to get a single 1024-d vector per 64x64 patch. This results in a 7x7x1024 tensor. Next, we use a PixelCNN-style autoregressive model [19] (a convolutional row-GRU PixelRNN [39] gave similar results) to make predictions about the latent activations in following rows top-to-bottom, visualized in Figure 4. We predict up to five rows from the 7x7 grid, and we apply the contrastive loss for each patch in the row. We used Adam optimizer with a learning rate of 2e-4 and trained on 32 GPUs each with a batch size of 16.

For the linear classifier trained on top of the CPC features we use SGD with a momentum of 0.9, a learning rate schedule of 0.1, 0.01 and 0.001 for 50k, 25k and 10k updates and batch size of 2048 on a single GPU. Note that when training the linear classifier we first spatially mean-pool the 7x7x1024 representation to a single 1024 dimensional vector. This is slightly different from [36] which uses a 3x3x1024 representation without pooling, and thus has more parameters in the supervised linear mapping (which could be advantageous).

Tables 3 and 4 show the top-1 and top-5 classification accuracies compared with the state-of-the-art. Despite being relatively domain agnostic, CPCs improve upon state-of-the-art by 9% absolute in top-1 accuracy, and 4% absolute in top-5 accuracy.

### 3.3 Natural Language

Our natural language experiments follow closely the procedure from [26] which was used for the skip-thought vectors model. We first learn our unsupervised model on the BookCorpus dataset [42], and evaluate the capability of our model as a generic feature extractor by using CPC representations for a set of classification tasks. To cope with words that are not seen during training, we employ vocabulary expansion the same way as [26], where a linear mapping is constructed between word2vec and the word embeddings learned by the model.

For the classification tasks we used the following datasets: movie review sentiment (MR) [43], customer product reviews (CR) [44], subjectivity/objectivity [45], opinion polarity (MPQA) [46] and question-type classification (TREC) [47]. As in [26] we train a logistic regression classifier and

| Method | Top-1 ACC |
|---|---|
| **Using AlexNet conv5** | |
| Video [28] | 29.8 |
| Relative Position [11] | 30.4 |
| BiGan [35] | 34.8 |
| Colorization [10] | 35.2 |
| Jigsaw [29] * | 38.1 |
| **Using ResNet-V2** | |
| Motion Segmentation [36] | 27.6 |
| Exemplar [36] | 31.5 |
| Relative Position [36] | 36.2 |
| Colorization [36] | 39.6 |
| **CPC** | **48.7** |

Table 3: ImageNet top-1 unsupervised classification results. *Jigsaw is not directly comparable to the other AlexNet results because of architectural differences.

| Method | Top-5 ACC |
|---|---|
| Motion Segmentation (MS) | 48.3 |
| Exemplar (Ex) | 53.1 |
| Relative Position (RP) | 59.2 |
| Colorization (Col) | 62.5 |
| Combination of MS + Ex + RP + Col | 69.3 |
| **CPC** | **73.6** |

Table 4: ImageNet top-5 unsupervised classification results. Previous results with MS, Ex, RP and Col were taken from [36] and are the best reported results on this task.

| Method | MR | CR | Subj | MPQA | TREC |
|---|---|---|---|---|---|
| Paragraph-vector [40] | 74.8 | 78.1 | 90.5 | 74.2 | 91.8 |
| Skip-thought vector [26] | 75.5 | 79.3 | 92.1 | 86.9 | 91.4 |
| Skip-thought + LN [41] | 79.5 | 82.6 | 93.4 | 89.0 | - |
| CPC | 76.9 | 80.1 | 91.2 | 87.7 | 96.8 |

Table 5: Classification accuracy on five common NLP benchmarks. We follow the same transfer learning setup from Skip-thought vectors [26] and use the BookCorpus dataset as source. [40] is an unsupervised approach to learning sentence-level representations. [26] is an alternative unsupervised learning approach. [41] is the same skip-thought model with layer normalization trained for 1M iterations.

evaluate with 10-fold cross-validation for MR, CR, Subj, MPQA and use the train/test split for TREC. A L2 regularization weight was chosen via cross-validation (therefore nested cross-validation for the first 4 datasets).

Our model consists of a simple sentence encoder $g_{enc}$ (a 1D-convolution + ReLU + mean-pooling) that embeds a whole sentence into a 2400-dimension vector $z$, followed by a GRU (2400 hidden units) which predicts up to 3 future sentence embeddings with the contrastive loss to form $c$. We used Adam optimizer with a learning rate of 2e-4 trained on 8 GPUs, each with a batch size of 64. We found that more advanced sentence encoders did not significantly improve the results, which may be due to the simplicity of the transfer tasks (e.g., in MPQA most datapoints consists of one or a few words), and the fact that bag-of-words models usually perform well on many NLP tasks [48].

Results on evaluation tasks are shown in Table 5 where we compare our model against other models that have been used using the same datasets. The performance of our method is very similar to the skip-thought vector model, with the advantage that it does not require a powerful LSTM as word-level decoder, therefore much faster to train. Although this is a standard transfer learning benchmark, we found that models that learn better relationships in the childeren books did not necessarily perform better on the target tasks (which are very different: movie reviews etc). We note that better [49, 27] results have been published on these target datasets, by transfer learning from a different source task.